

Nombre del proyecto:

AirGuard – Portal Web de Calidad del Aire y Salud Ambiental

Fase:

2 – Integración y Procesamiento

Equipo:

Grupo SN-2

Tutor:

Carlos Guillermo Rodríguez

Integrantes y roles:

Alva Danila Hernández Turcios – Líder de proyecto Jr

Blanca del Carmen Mezquita Cornejo – QA / Documentación Jr.

Wilmer Daniel Martínez Rosas – Frontend Jr.

Walter Edgardo Rodríguez Valle – GIS Jr.

Francisco Eliseo Márquez Vásquez –Backend Jr.

Fecha de entrega:

19 de Julio del 2026

Índice

Resumen Ejecutivo

1. Fase 2 Consumo de Api

- 1.1 Probar endpoints de la Api elegida
- 1.2 Identificar campos relevantes
- 1.3 Tabla de requisitos formales
- 1.4 Implementar script o módulo inicial de consumo
- 1.5 Registrar hallazgos y limitaciones en bitácora

2. Fase 2 Procesamiento y almacenamiento

- 2.1 Limpiar y normalizar datos
- 2.2 Definir estructura de persistencia o caché
- 2.3 Ejemplo de documento MongoDB
- 2.4 Validar coordenadas, contaminantes y fechas
- 2.5 Matriz de validación
- 2.6 Documentar flujo técnico

Conclusión

Bitácora

Resumen Ejecutivo

Durante la Fase 2 del proyecto AirGuard se implementó la integración entre el sistema y la API pública OpenAQ v3, (Actualmente se utiliza la versión 3) con el propósito de obtener información ambiental en tiempo real sobre la calidad del aire. La información obtenida es procesada por el backend desarrollado en Express.js, validada y normalizada antes de ser almacenada en MongoDB Atlas.

El objetivo principal será garantizar que el sistema pueda consumir información ambiental en tiempo real, transformarla en registros normalizados y almacenarla de manera persistente para su posterior consulta y visualización en el portal web.

1. Fase 2: Consumo de API

Integrar una API pública de calidad del aire para obtener datos ambientales que serán utilizados por el sistema AirGuard en la generación de mapas, gráficas e indicadores de calidad del aire.

Como fuente principal de información ambiental se seleccionó la API pública OpenAQ, la cual ofrece datos provenientes de estaciones de monitoreo distribuidas en diferentes países.

La integración fue implementada en el backend mediante el módulo openaq.js, encargado de establecer la comunicación con la API utilizando solicitudes HTTPS y procesar las respuestas obtenidas en formato JSON.

Las funciones implementadas en este módulo incluyen:

- Establecer la conexión segura con OpenAQ.
- Realizar solicitudes HTTP GET a los diferentes endpoints.
- Obtener información de estaciones y mediciones ambientales.

- Recibir y validar respuestas en formato JSON.
- Gestionar errores de comunicación.
- Enviar los datos al módulo de procesamiento.

Además, el proyecto incorpora un servicio denominado scheduler.js, responsable de ejecutar automáticamente la consulta a OpenAQ cada 90 segundos para mantener actualizada la información almacenada en la base de datos.

1.1 Probar endpoints de la API elegida.

Las pruebas del backend se realizaron sobre los endpoints principales de OpenAQ v3, enfocados en la obtención de estaciones de monitoreo y mediciones por rangos temporales. El consumo real se realiza mediante peticiones seguras utilizando el módulo nativo node-fetch.

Se realizaron pruebas funcionales sobre los principales servicios ofrecidos por OpenAQ con el objetivo de comprobar su disponibilidad y verificar la estructura de información ambiental. Mediante solicitudes HTTPS se comprobó que la API devuelve datos en formato JSON con información sobre estaciones de monitoreo, ubicación, contaminantes y fechas de medición. Posteriormente, estos datos son almacenados en MongoDB Atlas.

Resultados esperados:

- API responde correctamente mediante HTTPS.
- Las respuestas son en formato JSON.
- Verificación del funcionamiento de la API.
- Confirmación de la disponibilidad de los datos.
- Comprobar la correcta comunicación del Backend.

Ejemplo de no hay mediciones recientes:

```
{  
  "_id": {  
    "$oid": "6a45daa42e0f440a6478457a"  
  },  
  "station_id": "2812571",  
  "timestamp": {  
    "$date": "2026-07-02T02:03:30.000Z"  
  },  
  "pollutants": {  
    "pm25": 0,  
    "pm10": 0,  
    "co": 0,  
    "no2": 0,  
    "o3": 0,  
    "so2": 0  
  },  
  "aqi": {  
    "value": 0,  
    "category": "Sin datos",  
    "color": "#bdbdbd",  
    "dominant_pollutant": null  
  },  
  "recommendation": "No hay mediciones recientes disponibles para esta estación."  
}
```

1.2 Identificar campos relevantes.

Después de analizar la respuesta de la API, se identificaron los campos que serán utilizados por el sistema AirGuard.

Con el fin de evitar sobrecargar la base de datos MongoDB Atlas y optimizar las consultas del frontend, se seleccionaron únicamente los metadatos y valores indispensables para la visualización en el mapa cartográfico.

La siguiente tabla presenta el mapeo de campos procesados por el backend:

Campo	Utilidad
Estación	Identificar el punto de monitoreo.
País	Ubicación geográfica.
Ciudad	Filtrar información por ciudad.
Latitud y Longitud	Mostrar la ubicación en el mapa.
Contaminante	Identificar el tipo de medición.
Valor	Nivel registrado del contaminante.
Unidad	Interpretar correctamente la medición.
Fecha y hora	Registrar el momento de la medición.

1.3 Tabla de requisitos formales

Para asegurar un consumo estable y cumplir con los lineamientos técnicos del proyecto AirGuard.

Para asegurar un consumo estable y cumplir con los lineamientos técnicos definidos por el consorcio del proyecto AirGuard, se definieron requisitos formales.

ID	Requisito	Tipo	Prioridad	Fuente/Control	Estado
RF-01	Consumo de API OpenAQ v3 mediante peticiones asíncronas no bloqueantes en Node.js.	Funcional	Alta	Módulo openaq.js	Implementado
RF-02	Manejo de Rate Limits (límites de consumo) controlando reintentos exponenciales automáticos.	Funcional	Alta	Módulo openaq.js	Implementado
RF-03	Mostrar estaciones en mapa interactivo con Leaflet	Funcional	Alta	El mapa mostrará puntos con latitud/longitud y color según categoría ICA	Prototipo
RF-04	Calcular categoría ICA automáticamente	Funcional	Media	Cada registro incluirá categoría AQI (Buena,	Implementado



				Moderada, Riesgo alto)	
RF-05	Validación estricta de estructura JSON entrante previo a su almacenamiento.	Funcional	Media	Controlador Backend	Implementado
RNF-01	Proteger claves y configuración	No funcional	Alta	Ninguna credencial aparecerá en GitHub, se usan variables de entorno	Implementado
RNF-02	Actualizar datos periódicamente	No funcional	Media	El scheduler ejecutará consultas cada 90 segundos y actualizará la base de datos	Implementado

1.4 Implementar script o módulo inicial de consumo.

El desarrollo del backend del proyecto estructuró un módulo exclusivo denominado openaq.js. Este componente encapsula toda la interacción con la API externa, garantizando una arquitectura modular limpia y escalable.

Su implementación evita el acoplamiento con las rutas express.

Funciones principales dentro de openaq.js:

discoverStations(): Consulta la API de OpenAQ para descubrir estaciones de medición válidas, registrando o actualizando las que ya existen en nuestra colección local. Filtra por disponibilidad de parámetros y consistencia de coordenadas geográficas.

fetchBatchMeasurements(): Solicita mediciones de forma masiva para un grupo de estaciones dadas. Filtra las lecturas corruptas (valores negativos o nulos) y las devuelve normalizadas según nuestro modelo de datos interno.

Código de Implementación Base (openaq.js):

Extracto de openaq.js (Módulo de Consumo)

```
const fetch = require('node-fetch'); // Native fetch en Node.js 18+
const OPENAQ_API_URL = 'https://api.openaq.org/v3';
const OPENAQ_API_KEY = process.env.OPENAQ_API_KEY;
async function fetchBatchMeasurements(locations = []) {
  try {
    const response = await
    fetch(`${OPENAQ_API_URL}/measurements?limit=100`, {
      headers: { 'X-API-Key': OPENAQ_API_KEY }
    });
    if (!response.ok) throw new Error(`HTTP Error: ${response.status}`);
    const data = await response.json();
    return data.results;
  } catch (error) {
    console.error('Error fetching OpenAQ measurements:', error.message);
    return null; // Retorna null para disparar el modo de respaldo
  }
}
```

Se desarrolló un módulo openaq.js responsable de centralizar todas las operaciones relacionadas con la comunicación inicial en el backend encargado de realizar solicitudes a la API OpenAQ y obtener la información necesaria para el procesamiento de los datos.

Las funciones principales del módulo serán:

- Configuración del cliente HTTP.
- Establecer conexión con la API.
- Obtener la respuesta en formato JSON.
- Validar la información recibida.
- Manejo de errores de comunicación.
- Envío de los datos al módulo de procesamiento.

1.5 Registrar hallazgos y limitaciones en bitácora.

Durante las pruebas iniciales se identificaron algunos aspectos que deberán considerarse durante el desarrollo del proyecto.

Hallazgos

- La API proporciona información en formato JSON.
- Es posible consultar diferentes contaminantes.
- Los datos incluyen coordenadas geográficas para su representación en mapas.

Limitaciones

- Algunas estaciones presentan registros incompletos.
- Actualmente OpenAQ no dispone de estaciones para El Salvador.
- No todas las ubicaciones registran todos los contaminantes.
- La disponibilidad de datos depende de las estaciones de monitoreo.
- Debido a que el Ministerio de Medio Ambiente y Recursos Naturales (MARN) de El Salvador actualmente no cuenta con una API pública que

Permite acceder a la información ambiental de forma automatizada, para este proyecto se trabajará con un conjunto de datos de demostración (datos demo).

2. Fase 2: Procesamiento y almacenamiento

Garantizar la calidad e integridad de la información obtenida desde OpenAQ mediante procesos de validación, limpieza y normalización antes de su almacenamiento permanente.

2.1 Limpiar y normalizar datos.

Los datos obtenidos son sometidos a un proceso de depuración antes de ser almacenados.

Las actividades realizadas incluyen:

- Eliminación de registros duplicados.
- Validación de campos obligatorios.
- Conversión de fechas al estándar ISO 8601.
- Normalización de nombres de contaminantes.
- Verificación de unidades de medida.
- Validación de valores numéricos.
- Eliminación de registros incompletos o inconsistentes.

Este procedimiento garantiza la consistencia de la información utilizada por el sistema.

El objetivo es transformar los "jeroglíficos técnicos" de las APIs en registros legibles. Cada registro procesado debe incluir obligatoriamente los siguientes campos normalizados.

Temporales: Fecha y hora de la medición en formato local.

Geográficos: Nombre de la zona o estación y sus coordenadas geográficas (Latitud/Longitud).

Técnicos: Tipo de contaminante (priorizando PM2.5 y PM10), valor numérico y unidad de medida ($\mu\text{g}/\text{m}^3$).

Interpretativos: Una Categoría ICA (Índice de Calidad del Aire) calculada automáticamente, que asigne un color y un mensaje de salud (ej. "Buena", "Moderada", "Riesgo alto") según el valor detectado.

Metadatos: Fuente original de la información (ej. MARN / OpenAQ).

2.2 Definir estructura de persistencia o caché.

Se propone una arquitectura que soporte tanto el análisis histórico como la eficiencia del portal.

Persistencia (Base de Datos): Se recomienda el uso de MongoDB Atlas para guardar el histórico de lecturas, permitiendo visualizar tendencias temporales y comparativas entre zonas.

Almacenamiento Ligero: Como alternativa o respaldo, se pueden utilizar archivos JSON o Google Sheets para almacenar datos procesados de forma inmediata.

Optimización: El sistema debe actuar como un almacén intermedio que soporte las consultas del frontend, evitando depender exclusivamente de llamadas repetitivas a la API externa para no exceder los límites de consumo.

Para el almacenamiento permanente de la información se utiliza MongoDB Atlas. La colección principal del sistema es Measurements (Mediciones), diseñada para almacenar cada una de las mediciones obtenidas desde OpenAQ.

Los principales atributos definidos son:

- Identificador del registro.
- Estación de monitoreo.
- País.
- Ciudad.
- Latitud.
- Longitud.
- Contaminante.
- Valor de la medición.
- Unidad de medida.
- Fecha de la medición.
- Categoría del Índice de Calidad del Aire (ICA/AQI).
- Fecha de actualización del registro.

Esta estructura permite realizar consultas eficientes para la generación de mapas, indicadores y estadísticas.

2.3 Ejemplo de documento en MongoDB Atlas

A continuación, se presentan dos documentos reales que representan la persistencia en producción dentro de MongoDB Atlas.

El primero corresponde a un registro simulado localmente y el segundo a un registro recuperado y procesado de la API de OpenAQ:

Ejemplo 1: Registro Simulado de Calidad de Aire (El Salvador)

```
{
  "_id": ObjectId("6695b2c1f9b3e1201ab78cd1"),
  "station_id": "SV-SS-01",
  "country": "SV",
  "city": "San Salvador",
  "location": { "type": "Point", "coordinates": [-89.2182, 13.6929] },
  "parameter": "PM2.5",
  "value": 12.5,
  "unit": "µg/m³",
  "date": ISODate("2026-07-13T20:30:00Z"),
  "aqi_category": "Buena",
  "last_updated": ISODate("2026-07-13T20:31:00Z")
}
```

Ejemplo 2: Registro de Estación Física Procesado de OpenAQ

```
{
  "_id": ObjectId("6695b2c1f9b3e1201ab78cd2"),
  "station_id": "US-EPA-840060309",
  "country": "US",
  "city": "Los Angeles",
  "location": { "type": "Point", "coordinates": [-118.2437, 34.0522] },
  "parameter": "O3",
  "value": 0.054,
  "unit": "ppm",
  "date": ISODate("2026-07-13T19:45:00Z"),
  "aqi_category": "Buena",
  "last_updated": ISODate("2026-07-13T20:31:00Z")
}
```

2.4 Validar coordenadas, contaminantes y fechas.

El servicio de integración debe implementar las siguientes validaciones críticas antes de almacenar o mostrar la información.

Coordenadas:

Verificar la existencia de datos de latitud y longitud para asegurar que los puntos se ubiquen correctamente en el mapa interactivo (Leaflet).

Contaminantes:

Filtrar y validar que los parámetros correspondan a los contaminantes criterios definidos (PM2.5, PM10) o adicionales disponibles (CO, NO₂, O₃).

Fechas:

Asegurar que los registros sigan un formato estándar y correspondan a períodos válidos para el análisis de tendencias de las últimas 24 horas o históricos.

Control de errores:

Implementar lógica para manejar escenarios de datos incompletos o API sin respuesta. Antes del almacenamiento definitivo cada registro es sometido a un conjunto de validaciones automáticas.

Las principales validaciones incluyen:

- Verificación de coordenadas geográficas.
- Validación de contaminantes reconocidos por el sistema.
- Comprobación del formato de fecha.
- Validación de valores numéricos.
- Verificación de unidades de medida.

- Eliminación de registros duplicados.
- Confirmación de integridad del registro.

2.5 Matriz de validación

Con el fin de evaluar la resiliencia del sistema AirGuard ante fallas del entorno, variaciones en los formatos de la API o valores extremos, se diseñó e implementó la siguiente matriz de casos de prueba de validación:

Caso	Validación	Resultado esperado	Evidencia (captura/log/documento)	Resultado real
CV-01	Petición HTTP exitosa a OpenAQ v3.	Retorna JSON de estaciones y mediciones.	Script en GitHub /Backend	Exitoso – API responde con código 200 y datos JSON válidos
CV-02	Error de red, caída o timeout de API OpenAQ.	Registra error de comunicación en log.	Script en GitHub/Backend	Controlado – Sistema registra error en logs y activo modo simulador
CV-03	Medición PM2.5 = 25.4 $\mu\text{g}/\text{m}^3$ (Rango Moderado).	Se clasifica como “Sensible” en ICA	Script en GitHub/Backend	Exitoso – Registro clasificado como 'Moderada' (ICA 51-100)

CV-04	Clave de API de OpenAQ incorrecta o nula.	El sistema no expone credenciales y muestra error	Script en GitHub/Backend	Controlado – Error capturado, variable de entorno requerida
CV-05	Medición corrupta o valor negativo (ej. PM10 = -15).	El sistema descarta la lectura de inmediato.	Script en GitHub/Backend	Exitoso – Registro descartado y excluido del almacenamiento
CV-06	Scheduler ejecuta cada 90 segundos	La base de datos se actualiza automáticamente	Script en GitHub/Backend	Exitoso – Scheduler ejecutó correctamente, BD actualizado cada 90s

2.6 Documentar flujo técnico.

El flujo operativo del sistema sigue este orden lineal:



1. Ingestión: Consumo de datos crudos desde la API pública ambiental (OpenAQ).
2. Transformación: Limpieza de registros y cálculo del indicador de salud ambiental (ICA) mediante un script en el backend (Node.js).
3. Almacenamiento: Envío de los datos ya normalizados a la base de datos (MongoDB).
4. Exposición: El backend sirve estos datos procesados mediante una API propia para que el portal web los consuma.
5. Visualización: Representación de la información en el portal mediante mapas por zonas y gráficos de tendencias.

Esta fase es fundamental para lograr el propósito social del proyecto transformar datos técnicos dispersos en decisiones informadas de salud para el ciudadano.

Este flujo garantiza que únicamente información validada llegue a los usuarios finales.

Etapas	Descripción
1. API OpenAQ	Fuente de datos ambientales en tiempo real (estaciones, contaminantes, coordenadas y fechas de medición).
2. Consumo de datos	El módulo backend, realiza solicitudes HTTP a la API y obtiene la respuesta en formato JSON.
3. Validación	Se verifica que cada registro tenga coordenadas válidas, contaminante reconocido, fecha con formato correcto y valores numéricos completos.
4. Limpieza y normalización	Se eliminan registros duplicados o incompletos y se estandarizan nombres de contaminantes y unidades de medida.
5. MongoDB Atlas	Los registros validados y normalizados se almacenan de forma persistente en la colección "Mediciones".
6. Backend	Expone los datos ya procesados a través de una API interna para su consumo por el frontend.
7. Portal Web AirGuard	Consume la API interna del backend para generar mapas, gráficas e indicadores de calidad del aire.

Conclusión

La Fase 2 permitió consolidar la integración técnica entre el sistema AirGuard y la API pública OpenAQ v3, logrando un consumo automático de datos ambientales en tiempo real mediante un backend desarrollado en Express.js. Los registros obtenidos fueron sometidos a procesos de validación, limpieza y normalización, garantizando su calidad antes de ser almacenados en la colección *Measurements* de MongoDB Atlas.

Con esta infraestructura, el proyecto asegura que la información ambiental se procese de manera confiable y esté disponible para su posterior consulta y visualización en el portal web.

Esto deja preparado el terreno para la Fase 3, enfocada en la construcción del dashboard, la integración de mapas interactivos y la generación de indicadores visuales que acerquen la información de calidad del aire al ciudadano.

Bitácora

ESIT - ESTANCIA PROFESIONAL

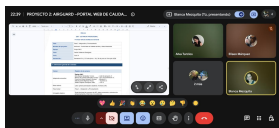
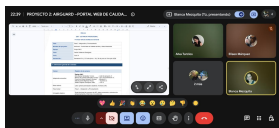
Formato Oficial de Bitácora Semanal

Fase:	Fase 2: Integración y Procesamiento
Nombre del proyecto:	AirGuard – Portal Web de Calidad del Aire y Salud Ambiental
Equipo:	Grupo SN-2
Tutor:	Carlos Guillermo Rodríguez
Ciclo:	Ciclo VI
Semana No.:	Semanas N° 4 y 5 Fecha (inicio - fin): 29 de junio al 19 de julio 2026

1. Información general de la semana

Campo	Registro de la semana
Integrantes presentes	Equipo SN-2 Alva Danila Hernández Turcios – Líder de proyecto Jr Blanca del Carmen Mezquita Cornejo – QA / Documentación Jr. Francisco Eliseo Márquez Vásquez – Backend Jr. Wilmer Daniel Martínez Rosas – Frontend Jr. Walter Edgardo Rodríguez Valle – GIS Jr.
Roles activos	Líder, QA/Documentación, Backend, Frontend, GIS.
Fase Actual	Fase 2 – Integración y Procesamiento
Entregable objetivo	Script funcional de consumo de API, datos procesados, estructura de almacenamiento y bitácoras con evidencias.
Acuerdos clave	Uso de OpenAQ v3 como API principal, MongoDB Atlas como base de datos, normalización de campos según modelo definido en Fase 1.
Riesgos iniciales y Estado General	Límites de consumo de API o datos incompletos, además estado es avance satisfactorio, pruebas iniciales completadas.

2. Actividades realizadas

Actividad	Descripción técnica	Responsable	Evidencia verificable
Consumo de API OpenAQ v3	Script inicial para extracción de datos ambientales.	Francisco	Captura de script en GitHub
Configuración del módulo OpenAQ	Desarrollo del módulo openaq.js para consumir la API.	Francisco	Código fuente
Pruebas de conexión	Validación de respuestas HTTP y formato JSON.	Francisco/Wilmer	Captura
Identificación de campos	Definición de atributos requeridos para el sistema.	Blanca	Documento técnico 
Diseño del flujo	Definición del proceso de validación, limpieza y almacenamiento	Walter	Diagrama del flujo
Diseño de MongoDB Atlas	Definición de la colección Measurements	Francisco/Alva	Esquema
Revisión de arquitectura	Validación de la arquitectura	Equipo SN-2	N/A
Actualización documental y registro en bitácora	Elaboración y consolidación del informe y bitácoras.	Blanca	Informe de fase 2 con sus respectivas bitácora actualizadas 

3. Herramientas y servicios utilizados

Herramienta / servicio	Propósito	Observaciones
OpenAQ v3	Fuente de información ambiental	API Módulo de conexión en Node.js.
MongoDB Atlas	Almacenamiento de registros.	Configuración básica completada.
Visual Studio Code	Desarrollo	Editor
Leaflet	Mapas interactivos	Validación de coordenadas
Docker	Despliegue multiplataforma	Imagen inicial creada
GitHub	Control de versiones.	Repositorio activo

4. Problemas detectados y análisis

Problema	Impacto	Causa probable	Evidencia
Ausencia de estaciones OpenAQ en El Salvador	No se pueden mostrar datos locales del país.	Cobertura limitada de la red OpenAQ en la región.	Consulta a la API sin resultados para El Salvador.
Información incompleta en algunas estaciones	Riesgo de registros con campos vacíos.	Sensores limitados o mantenimiento irregular de estaciones.	Revisión de respuestas JSON con campos nulos.

5. Soluciones aplicadas

Solución aplicada	Problema asociado	Responsable	Evidencia
Uso de otros países/ciudades con cobertura disponible como referencia	Ausencia de estaciones en El Salvador	Walter	Consulta a estaciones con cobertura OpenAQ

Solución aplicada	Problema asociado	Responsable	Evidencia
Validación de campos obligatorios antes del almacenamiento	Información incompleta	Francisco	Reglas de validación del módulo de consumo

6. Estado del proyecto esta semana

Área	Estado	Comentarios
Fase 2 – Integración y Procesamiento	Completado	Script funcional, datos normalizados, repositorio y bitácoras actualizados.

7. Plan de trabajo para la próxima semana

Tarea próxima semana	Responsable	Fase	Entregable esperado
Construcción dashboard	Francisco	Fase 3	Portal inicial funcional con datos integrados.
Integración Leaflet	Walter	Fase 3	Mapas y visualización analítica
Indicadores y pruebas	Wilmer	Fase 3	Matriz de pruebas, resultados.
Actualizar bitácora y evidencias	Blanca	Fase 3	Documento técnico y bitácora actualizada.
Coordinación y asignación de actividades	Alva	Fase 3	Google meet

8. Aportes individuales verificables

Integrante	Rol	Tarea técnica específica realizada	Componente / entregable afectado	Evidencia verificable	Resultado
Alva	Líder Jr.	Coordinación y seguimiento	Planificación	Reuniones	
Blanca	QA/Documentación Jr.	Redacción de Fase 2, bitácora y documentación.	Documentación	Documentación y bitácoras.	Evidencias consolidadas. 
Francisco	Backend Jr.	Consumo de API y normalización de datos	Backend	Script en GitHub	API integrada
Wilmer	Frontend Jr.	Script de integración con vistas iniciales	Frontend	Commit en GitHub	Prototipo
Walter	GIS Jr.	Validación de coordenadas y fuentes de datos	GIS	Diagramas	Prototipo

9. Cumplimiento individual y responsabilidad

Integrante	Tarea asignada	Estado	Observación del cumplimiento	Seguimiento requerido
Alva	Coordinación del proyecto	Completado	Roles asignados	Supervisar Fase 3
Blanca	Bitácoras y documentación técnica	Completado	Documentación organizada	Mantener actualizaciones
Francisco	Consumo de API	Completado	Script operativo	Preparar persistencia
Wilmer	Script de integración	Completado	Script funcional	Iniciar diseño de interfaz y prototipo
Walter	Validación GIS	Completado	Fuentes seleccionadas	Continuar análisis y prototipo

10. Participación en documentos técnicos y entregables

Integrante	Documento / entregable	Sección elaborada o corregida	Tipo de aporte	Evidencia
Alva	Responsable de entregar Fase 2	Planificación	Coordinación	Documento PDF
Blanca	Informe completo de Fase 2	Documentación y bitácora	Redacción, edición, actualización y consolidación.	Documento entregable y bitácoras actualizadas en carpeta Drive https://drive.google.com/drive/folders/1ypgbOfJS1kgvW9kITPY9L_v1l6_IRrEa?usp=sharing
Francisco	Backend	Consumo de API	Técnica	Script en GitHub https://github.com/blancamezquita09-rgb/AirGuard.git
Wilmer	Script de integración	Vistas iniciales	Técnica	Commit en GitHub
Walter	Documento técnico	Validación GIS	Investigación	Carpeta Drive https://drive.google.com/drive/folders/1ypgbOfJS1kgvW9kITPY9L_v1l6_IRrEa?usp=sharing

11. Pruebas, configuraciones o validaciones por estudiante

Integrante	Tipo de prueba / configuración	Elemento intervenido	Resultado	Evidencia
Alva	Validación de comunicación	Meet y WhatsApp	Positivo	
Blanca	Configuración Drive	Carpeta compartida	Visualización correcta	 https://drive.google.com/drive/folders/1ypgbOfJS1kgvW9kITPY9Lv1l6_IJrEa?usp=sharing
Wilmer	Verificación integración	Script	Funcional	En proceso
Walter	Validación OpenAQ	Coordenadas	Positivo	Documento
Francisco	Configuración backend	Script	Correcto	Commit en GitHub https://github.com/blancamezquita09-rgb/AirGuard.git

12. Participación en demo o explicación técnica (Solo utilizar en entrega del informe final)

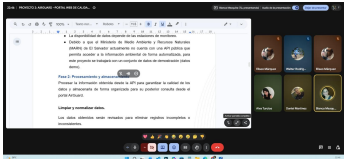
Integrante	Parte explicada o demostrada	Evidencia presentada	Nivel de dominio observado
No aplica en Fase 2	Se completará en Fase 4	N/A	N/A

13. Reflexión individual por rol

Integrante	Rol	Qué realizó	Dificultad técnica enfrentada	Cómo la resolvió	Próximo paso
Alva	Líder Jr.	Coordinó reuniones	Organización de tareas	Uso de Google calendar	Supervisar Fase 3
Blanca	QA/Documentación Jr.	Elaboró la documentación de la fase y actualizó la bitácora del proyecto.	Consolidar la información generada por todos los integrantes.	Organización de documentos compartidos mediante Google Drive.	Elaborar el informe de aseguramiento de calidad (QA) y validar la documentación final.
Wilmer	Frontend Jr.	Script de integración	Configuraciones	Comparaciones	Diseñar y optimizar la interfaz de usuario para la siguiente fase.
Walter	GIS Jr.	Validación de coordenadas	Inconsistencia en datos	Normalización	Integrar el mapa interactivo con los datos validados.
Francisco	Backend Jr.	Implementó el consumo de la API	Cambios en la estructura	Ajuste del script de consumo y	Implementar la persistencia de

Integrante	Rol	Qué realizó	Dificultad técnica enfrentada	Cómo la resolvió	Próximo paso
		OpenAQ y el procesamiento de datos.	de respuesta de la API.	transformación de datos.	datos y optimizar el procesamiento automático.

14. Registro consolidado de evidencias

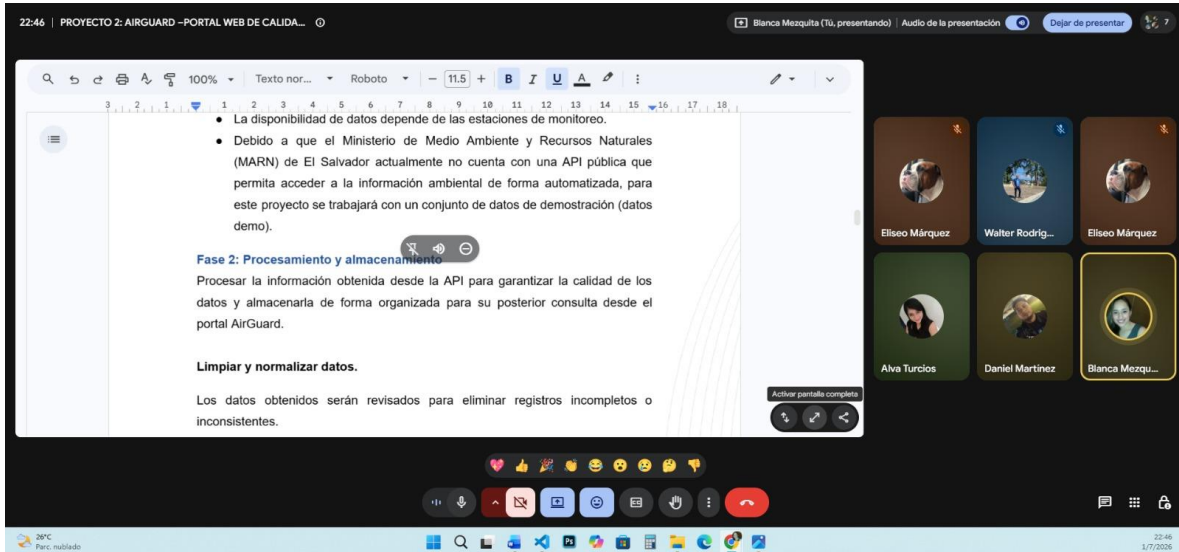
No.	Descripción de la evidencia	Tipo	Responsable	Enlace / ruta / referencia
1	Script inicial de consumo de API OpenAQ	Código fuente	Francisco	GitHub – Repositorio AirGuard https://github.com/blancam ezquita09-rgb/AirGuard.git
2	Documentos técnicos, bitácora actualizada del entregable Fase 2	Documento Oficial	Blanca	Carpeta Drive - Fase 2 Estancia Profesional 01-2026 Grupo 2
3	Calendario de reuniones en Google Calendar	Evidencia organizativa	Alva	

15.Registro de evidencias

Descripción de la evidencia:

Evidencia 1:

Captura de reunión de equipo en proceso de elaboración de Fase 2 (1 de julio del 2026 a las 10:30 p.m.)



Evidencia 2:

Carpeta compartida del proyecto AirGuard en Google Drive, utilizada para almacenar documentos, bitácoras, diagramas y evidencias generadas durante las fases del proyecto.

[https://drive.google.com/drive/folders/1ypgbOfJS1kgvW9kITPY9L_v1l6_IRrEa?usp=drive link](https://drive.google.com/drive/folders/1ypgbOfJS1kgvW9kITPY9L_v1l6_IRrEa?usp=drive_link)

Evidencia 3:

Prototipo que refleja la estructura visual y funcional del sistema antes de su implementación final.

Pantallas principales: Inicio, mapa, histórico y zonas.

Indicadores de calidad del aire: ICA, AQI máximo, contaminantes, recomendaciones según categoría y funciones de exportación y difusión.

AirGuard

InicioMapaHistoriaZonas

ICA

Calidad del aire

Recomendacion segun categoria

EstacionesAQI maxContaminante

Exportar PDF

WhatsApp

X/Twitter

Contaminantes

Promedio 5 estaciones

Contaminante 1

Contaminante 2

Contaminante 3

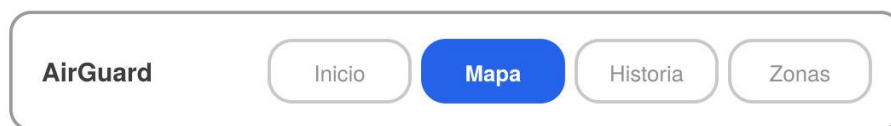
Contaminante 4

Contaminante 5

Contaminante 6

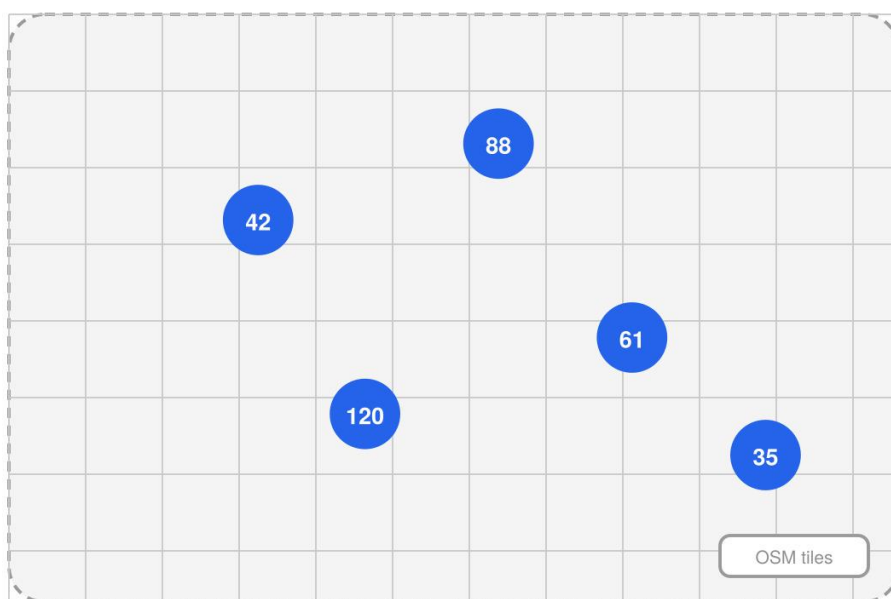
**Escuela
Profesional**
ESCUELA SUPERIOR DE INNOVACIÓN Y TECNOLOGÍA

Mapa geoespacial: Visualización con estaciones activas y valores numéricos.



Mapa geoespacial

Actualizado 12:41



Estaciones activas

	Estacion 1		
	Estacion 2		
	Estacion 3		
	Estacion 4		
	Estacion 5		

Histórico y comparación: Datos de las últimas 24 horas y comparaciones entre zonas.

AirGuard

Inicio

Mapa

Historia

Zonas

Historico

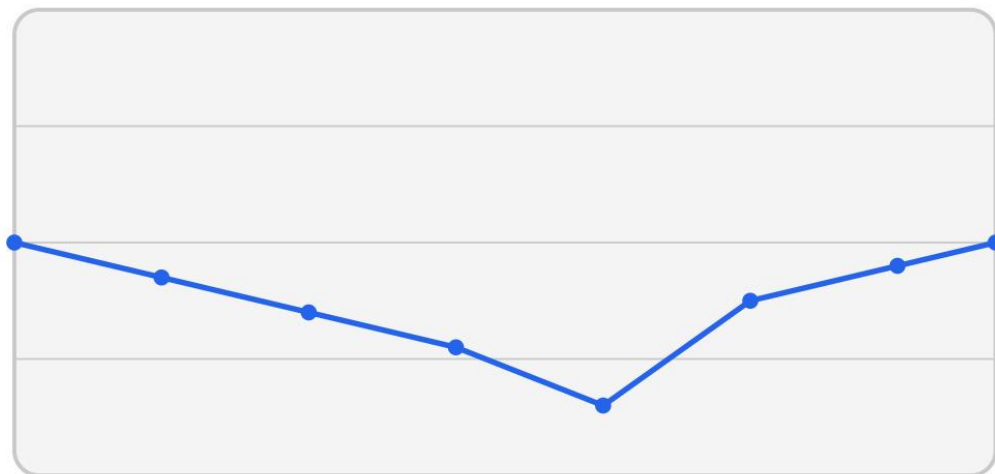
Contaminante 1

Contaminante 2

Contaminante 3

Contaminante 4

Contaminante 5



Ultimas 24 horas - San Salvador



Exportar reporte PDF

AirGuard

Inicio

Mapa

Historia

Zonas

Comparacion por zona

Comparacion por zona

Zona 1



Zona 2



Zona 3



Zona 4



Zona 5



Zona 6



Evidencia 4:

Se realizó una reunión virtual mediante Google Meet con los integrantes del Grupo SN-2 para presentar los avances de la Fase 2 del proyecto AirGuard. Durante la sesión se revisó la bitácora del proyecto, el cumplimiento de las actividades asignadas, la integración del consumo de la API, el procesamiento de datos y la organización de la documentación para la entrega correspondiente.

22:39 | PROYECTO 2: AIRGUARD –PORTAL WEB DE CALIDA... ① Blanca Mezquita (Tú, presentando)

The screenshot shows a Google Meet interface with a presentation window on the left and participant avatars on the right. The presentation is a document titled 'Bitácora EBIT - ESTANCIA PROFESIONAL Formato Oficial de Bitácora Semanal'. It contains the following information:

Fase 2: Integración y Procesamiento	
Nombre del proyecto:	AirGuard - Portal Web de Calidad del Aire y Salud Ambiental
Equipo:	Grupo SN-2
Tutor:	Carlos Guillermo Rodríguez
Ciclo:	Ciclo VI
Semana No.:	Semanas N° 4 y 5 Fecha (inicio - fin): 29 de junio al 19 de julio 2026

Below this, there is a section '1. Información general de la semana' with a table for 'Registro de la semana'.

Campo	Registro de la semana
Equipo SN-2	Alva Daniela Hernández Turcios - Líder de proyecto J. Blanca del Carmen Mezquita Coraggio - QA - Documentación J. Francisco Eliseo Márquez Viquez - Backend J. Walter Daniel Martínez Ríos - Frontend J. Walter Edgardo Rodríguez Valle - GIS J.
Integrantes presentes	
Roles activos	Líder: QA/Documentación, Backend, Frontend, GIS.
Fase Actual	Fase 2 - Integración y Procesamiento
Entregable objetivo	Script funcional de consumo de API, datos procesados, estructura de almacenamiento y bitácora con evidencias.

On the right side of the Meet window, there are four participant avatars: Alva Turcios, Eliseo Márquez, 2 más (2 more), and Blanca Mezquita (who is presenting).

22:39 | PROYECTO 2: AIRGUARD –PORTAL WEB DE CALIDA... ① Blanca Mezquita (Tú, presentando)

This screenshot shows the same Google Meet session, but with a different arrangement of participant avatars on the right. The presentation window on the left is identical to the previous screenshot, showing the 'Bitácora' document. The avatars now include Blanca Mezquita, 4 más (4 more), and Alva Turcios. The interface elements like the top bar and bottom controls remain the same.